Code: HTTP API layer (api.ts)

Project reference: STN22_Thoracic belt for respiratory rate measurement

Authors: Hakkou Dounia

Date: 2025–2026

File header

```
<!--
  Description: TypeScript module centralising all HTTP communication
  between the RespiraMed PWA frontend and the Node.js/Express backend.
  Exports two API objects (patientsAPI, sessionsAPI) and the Patient interface.
  Uses the native browser fetch() API with a shared helper function (apiRequest)
  that handles URL construction, JSON headers, and HTTP error propagation.

  Project reference: STN22_Thoracic belt FR
  Authors: Hakkou Dounia
  Date: 2025-2026
  © SIS TEAM NANCY
-->
```

API base URL

```
// Read the API base URL from the Vite environment variable.
// VITE_API_URL is defined in .env at project root.
// Fallback to the production URL if the variable is absent (e.g. in CI).
const API_URL = import.meta.env.VITE_API_URL
             || 'https://sisteamnancy.fr/api-respiramed';
```

Helper function: apiRequest()

```
// Generic HTTP request helper used by all API methods below.
// Parameters:
// endpoint: relative path, e.g. '/patients' or '/patients/42'
// options : optional RequestInit overrides (method, body, extra headers)
// Returns the parsed JSON response body.
// Throws an Error with the server message if the response is not OK.
async function apiRequest(endpoint: string, options: RequestInit = {}) {
  // Build the full request URL by concatenating the base URL and the endpoint
  const response = await fetch(`${API_URL}${endpoint}`, {
    ...options,
    headers: {
      // Always send JSON content-type so the backend parses the body correctly
      'Content-Type': 'application/json',
      // Merge any extra headers provided by the caller (e.g. auth tokens)
      ...options.headers,
    },
  });

  // If the HTTP status is outside the 2xx range, the request failed
  if (!response.ok) {
    // Try to extract an error message from the JSON body
    // If the body is not JSON, fall back to a generic message
    const error = await response.json()
      .catch(() => ({ message: 'Erreur serveur' }));
    throw new Error(error.message || 'Erreur lors de la requête');
  }

  // Deserialize and return the response body as a JavaScript object
  return response.json();
}
```

patientsAPI — Patient CRUD operations

```
// Object grouping all patient-related API calls.
// Each method delegates to apiRequest() with the appropriate HTTP verb.
export const patientsAPI = {
```

```
  // getAll
  // Sends GET /patients to retrieve the full list of registered patients.
  // Returns an array of Patient objects as stored in the MySQL database.
  async getAll() {
    return apiRequest('/patients');
  },

  // create
  // Sends POST /patients with the new patient data serialised as JSON.
  // The backend validates, inserts the record, and returns the created Patient
  // including the server-generated UUID and creation timestamp.
  async create(patient: {
    nom: string; prenom: string; age: number;
    sexe: string; taille: number; poids: number; telephone: string;
  }) {
    return apiRequest('/patients', {
      method: 'POST',
      body: JSON.stringify(patient),  // Serialise the object to a JSON string
    });
  },

  // delete
  // Sends DELETE /patients/:id where id is the patient's UUID.
  // The backend removes the patient record and all associated sessions/measurements.
  async delete(id: string) {
    return apiRequest(`/patients/${id}`, {
      method: 'DELETE',
    });
  },
};
```

sessionsAPI — Session lifecycle operations

```
// Object grouping session start/stop operations.
// A session represents one measurement period associated with a patient.
export const sessionsAPI = {

  // start
  // Sends POST /set-patient to register the active patient on the backend.
  // The server creates a new session record linked to patientId.
  // All subsequent LoRa measurements will be stored under this session.
  async start(patientId: string) {
    return apiRequest('/set-patient', {
      method: 'POST',
      body: JSON.stringify({ patientId }),
    });
  },

  // stop
  // Sends POST /stop-session to close the currently active session.
  // If sessionId is provided, only that specific session is closed.
  // If omitted, the backend closes whatever session is currently active.
  async stop(sessionId?: string) {
    return apiRequest('/stop-session', {
      method: 'POST',
      // Send an empty body when no sessionId is given
      body: JSON.stringify(sessionId ? { sessionId } : {}),
    });
  },
};
```

Patient interface

```
// TypeScript interface describing the shape of a Patient object
// as returned by the backend from the MySQL 'patients' table.
// All fields are required here because the backend always provides them.
export interface Patient {
  id: string;          // UUID generated by MySQL on insertion
  nom: string;         // Family name
  prenom: string;      // First name
  taille: number;      // Height in centimetres
```

```
  age: number;          // Age in full years
  sexe: string;         // Biological sex ('M', 'F', or 'Autre')
  telephone: string;    // Phone number
  poids: number;        // Weight in kilograms
  created_at: string;   // ISO 8601 creation timestamp set by the server
}
```